

Требования к разработке игровой программы

Данные требования предъявляются к разработке игровой программы на языке C++ с использованием подходов объектно-ориентированного программирования

Требования к проектированию классов

Нельзя использовать глобальные переменные

Использование глобальных функций допускается только при обосновании (например, при реализации шаблона Синглетон)

Нельзя использовать «дружественные» классы при проектировании.

Требования к интерфейсной части

Программа должна быть разработана с использованием библиотеки для разработки [FLTK](#).

FLTK — это кроссплатформенный инструментальный графического интерфейса C++ для UNIX®/Linux® (X11 и Wayland), Microsoft® Windows® и macOS®.

Студент самостоятельно должен изучить работу с данной библиотекой и использовать её для разработки графического интерфейса к игровой программе

Требования к публикации программы

Программа должна быть выложена в [GitVerse](#) в отдельном проекте.

Имя проекта должно быть **OOP_<номер группы>_<номер в журнале>_<фамилия студента на английском>**

Для сдачи проекта должен быть создан запрос на слияние (pull-request/merge-request) и назначен для проверки на преподавателя

Требования к игровой программе

Обязательным требованием является «аркадность» игры (игровые объекты должны быть в постоянном движении).

Другие варианты игры должны предварительно согласовываться с преподавателем.

Нельзя выбирать для разработки «элементарные» программы (крестики-нолики, пятнашки, 2048 и т.п.)

Требование к интерфейсу программы

- Обязательное наличие главного меню. В главном меню обязательно должны быть следующие пункты:
 - **Новая игра.** При выборе данного пункта должна запускаться новая игра
 - **О программе.** При выборе данного пункта должна показываться информация о программе, а также описание управления
 - **Выход.** При выборе данного меню программа должна завершаться.
- После окончания игры программа должна возвращаться к главному меню (именно возвращаться, а не пересоздавать окно с главным меню заново)

Соглашение по написанию кода

При написании программы необходимо использовать современный вариант соглашения по написанию кода (см. приложение). Использование соглашения, похожего на соглашение FLTK запрещено.

Требования к программе

- Использование шаблона Модель-Вид-Контроллер ([обобщённое описание](#))
 - Классы модели должны быть в отдельной директории проекта
 - Классы контроллера должны быть в отдельной директории проекта
 - Классы Вида должны быть в отдельной директории проекта
- Использование Объектно-Ориентированного подхода к проектированию (структура программы должна быть основана на классах)
- Использование всех видов отношений между классами (наследование, агрегация, ассоциация, композиция)
- Все поля классов должны быть закрытыми (private)
- Интеграция с FLTK
 - Использование статических функций для callback допускается только в классах View
 - Передача параметров типа void* в собственные разработанные методы допускается только для статических функций callback требуемых для интеграции с FLTK. Внутри собственных вызовов передача параметров с типом void* запрещена.
 - Использование типов и классов библиотеки FLTK допускается только в классах View
- Минимизировать работу с динамическими данными (создаваемых через new). Каждое использование динамических данных необходимо будет обосновать (вариант «потому что так было в примере не подходит»)
- Глобальных переменных и методов в программе быть не должно (исключение только для метода main)
- При работе с CallBack задавать метод для обратного вызова должен тот класс, чей метод будет вызываться. Не допускается задавать в качестве CallBack одним классом методов другого класса

Требования к проектированию структуры программы

Структура программы и иерархия классов должна быть спроектирована для возможности максимально простого расширения функционала программы:

- Добавление новых игровых объектов
- Добавление бонусных объектов

Требования к сборке программы

- При сборке все необходимые для работы программы файлы должны автоматически помещаться в папку сборки. Не допускается инструкций типа: «Для работы программы скопируйте такие-то файлы туда-то»

Требования к работе программы

- Соблюдение правил разрабатываемой игры
- Отсутствие ошибок/вылетания программы
- Отсутствие утечек памяти

Требования для сдачи программы

Программа должна быть готова к сдаче (выложена в git, сделан запрос на слияние и назначен на преподавателя) до начала зачётной недели.

Без выполнения всех требований программа не засчитывается (программа либо сделана по всем требованиям или не сделана).

Установка обработчиков callback

Устанавливать метод с callback должен тот класс, в котором метод объявлен (например, в класс MainView устанавливает callback на методы контроллера

Требования к реализации MVC

У каждого окна должен быть свой контроллер, который работает со своей моделью (если есть) и классом View

Классы модели ничего не должны знать про классы View (не допускается делать дружественные" объявления в модели для View

Классы View ничего не должны знать про классы контроллера

Контроллер должен получать действия пользователя сам (для текущего варианта программы - может получать через callback от View). Классы View ничего не должны знать про класс контроллера, так как у одного контроллера могут быть разные классы View для разных вариантов отображения модели

В классе контроллера не должно быть никакой логики (кроме создания нужных классов View и модели, а также логики по передаче действий пользователя в модель и взаимодействия с другими контроллерами).

Приложение

Общая информация

Присвоение имени файлу или переменной — это первый и самый простой шаг, который делает программист для написания чистого кода. Имя должно быть подходящим, чтобы любой другой программист мог легко прочитать код. В C++ соглашения об именовании — это набор правил для выбора допустимого имени переменной и функции в программе на C++.

- Имя класса должно быть существительным.
- Используйте заглавные буквы в качестве разделителей слов, а строчные — для остальной части слова в названии класса.
- Первый символ в имени класса должен быть написан заглавными буквами.
- В имени класса не допускаются подчеркивания ('_').
- Перед именем закрытого атрибута/поля в классе должен стоять префикс, само название начинаться с маленькой буквы
 - Перед статическими полями символ «s_»
 - Перед константами символ «c_» или написание только заглавными буквами через подчёркивание
 - Перед обычными полями символ «m_»
- После добавления префикса к названию будут применяться те же правила, что и к названию класса.
- Буква «m» также предшествует другим модификаторам имён. Например, «r» для указателей.
- Имя каждого метода / функции должно начинаться с глагола.
- Первый символ в именах аргументов функций/методов должен быть строчным. Все слова, начинающиеся после первой буквы, должны быть заглавными в именах классов.
- Имя переменной должно начинаться с алфавита.
- В имени переменной могут использоваться цифры, но только после алфавита.
- В именах переменных нельзя использовать специальные символы, кроме подчёркивания ('_').
- Для имен переменных нельзя использовать ключевые слова.
- К указательным переменным следует добавлять префикс «r» и ставить звёздочку «*» рядом с именем переменной вместо типа указателя.
- Ссылочным переменным следует добавлять префикс «r». Это помогает различать метод, возвращающий изменяемый объект, и тот же метод, возвращающий неизменяемый объект.
- Статические переменные следует предварять буквой "s".
- Все глобальные константы должны быть заглавными буквами, разделенными символом '_'.
_.
- В имени файла не должно быть специальных символов, кроме подчёркивания ('_') и тире ('-').
- Имя файла должно заканчиваться расширением .cc или .cpp.
- Не используйте имена файлов, которые уже существуют в /user/include. или любое другое предопределённое имя заголовочного файла.

Пример:

```
Вычислите произведение двух переменных-  
Назовите переменную l как var1
```

Назовите переменную 2 как var2
Назовите результат как произведение или pdt
Это будет более удобным соглашением об именовании, чем использование случайных имён вроде x, y и z.

Преимущества соглашений об именовании:

Соблюдение ряда правил [соглашения об именовании](#) при написании кода помогает в следующих случаях:

1. Позволяет избежать конфликтов именования.
2. Улучшает ясность кода в случае двусмысленности.
3. Помогает формализовать и продвигать согласованность внутри команды.
4. Улучшает понимание и читабельность кода.

Иллюстрации:

1. **Класс:** называйте класс в C++ в соответствии с его назначением и используйте заглавные буквы в качестве разделителей слов. Имя класса должно быть существительным. Например, ReverseString и Multiply2Numbers. Имя должно быть конкретным и намекать на функционирование класса, не заглядывая в его тело. Этот тип соглашения называется PascalCase.
2. **Методы:** каждый метод и функция выполняют какое-то действие, поэтому из названия функции должно быть понятно, что она делает. Например, writeDataToFile() — более удобное название, чем WriteFile(). Названия функций должны начинаться с глагола. Этот тип соглашения называется «верблюжий регистр». Мы также можем использовать «змеиный регистр» для методов. В «змеином регистре» мы соединяем два отдельных слова с помощью «_» (подчёркивания).
3. **Константы:** все константы должны быть написаны заглавными буквами с разделителями «_». Например, MAX_INT, TWO_PI и т. д.

Соглашение об именовании в C++

Имена в программе — ключ к её читабельности. Если имя в программе подходит, то всё встаёт на свои места, связи очевидны, а смысл понятен. В C++ для написания имён методов, переменных, классов, пакетов и констант используется CamelCase.

CamelCase — это соглашение об именовании, согласно которому имя состоит из нескольких слов, объединённых в одно слово с заглавной буквы.

Ниже приведены правила именования в программировании на C++. Их необходимо соблюдать при написании кода на C++ для удобства сопровождения, читабельности и понимания программы.

Тип 1: Названия классов и атрибутов классов

- Имя класса должно быть существительным.
- Используйте заглавные буквы в качестве разделителей слов, а строчные — для остальной части слова.
- Первый символ в имени класса должен быть написан заглавными буквами.
- В имени класса не допускаются подчёркивания ('_').

```
класс PerimeterRectangle  
класс FingerprintScanner
```

- Перед именем закрытого атрибута в классе должен стоять символ «m».
- После добавления «m» к названию будут применяться те же правила, что и к названию класса.

- Буква «т» также предшествует другим модификаторам имён. Например, «р» для указателей.

```
класс PerimeterRectangle
{
    private:
    int m_length;
    int m_width;
}
```

Тип 2: Функции и имена аргументов функций

Обычно каждая функция в C++ выполняет одно или несколько действий, поэтому название функции должно чётко указывать на то, что она делает. Название каждого метода/функции должно начинаться с глагола.

- Иногда полезны суффиксы. Например,
 - **Count** - текущее значение счетчика.
 - **Key**- значение ключа.
- Иногда полезны префиксы. Например,
 - **get**-получить значение.
 - **set**- установить значение.

Используется то же соглашение об именах, что и для имен классов, но имя должно начинаться с маленькой буквы.

```
int getValue();
int solvequation();
```

Первый символ в именах аргументов функций/методов должен быть строчной буквой. Каждое слово также должно начинаться с заглавной буквы.

```
int perimeterRectangle(int lengthRectangle, int widthRectangle)
```

Тип 3: Переменные

Когда переменные объявляются динамически с помощью ключевого слова **new** или когда переменные объявляются как атрибуты класса, они занимают память из кучи, а когда переменные создаются в программе на C++, память выделяется из стека программы.

- Имя переменной должно начинаться с алфавита.
- Цифры могут использоваться, но только после алфавита.
- В именах переменных нельзя использовать специальные символы, кроме подчёркивания ('_').
- Для имен переменных нельзя использовать ключевые слова.

```
int total_cost;
```

длина int;

Указателям на переменные следует добавлять префикс «р» и ставить звёздочку «*» рядом с именем переменной вместо типа указателя.

```
int *pName;
```

*int *pAge, адрес; // Здесь только pAge является переменной-указателем*

Ссылочные переменные должны начинаться с «r». Это помогает различать метод, возвращающий изменяемый объект, и тот же метод, возвращающий неизменяемый объект.

Статические переменные следует предварять буквой 's'.

статический int sCount;

Инициализация переменных, полей классов, структур

Все присвоения значений переменным должны делаться через инициализаторы (фигурные скобки). Это же требование актуально для инициализации конструкторов

Тип 4: Константы

Все глобальные константы должны быть заглавными буквами, разделенными символом `'_'`.

const double TWO_PI = 6,28318531;

Тип 5: Именованние файлов;

- В имени файла не должно быть специальных символов, кроме подчёркивания (`'_'`) и тире (`'-'`).
- Имя файла должно заканчиваться расширением `.c` или `.cpp`.
- Не используйте имена файлов, которые уже существуют в `/user/include.` или любое другое предопределённое имя заголовочного файла.

helloWorld.c // Действительный

helloWorld.cpp // Действительный

*hello * world.cpp // Недействительно*

iostream.cpp // Недействительно

hello123@world.cpp // Недействительно